NeverFull-NeverStop

A Transparent Context-Compaction Proxy for Long-Horizon Agents

V11 Project Report

Project maintainer `061115xhsm`

October 26, 2026 Version V11

Abstract

The bottleneck for long-horizon LLM agents is rarely single-turn reasoning; it is the context window. Once history outgrows the window, an agent either stops or silently loses state. Existing approaches—agent-native compaction (OpenClaw, Claude Code), runtime compressors (Hermes), and engineered memory layers—trade off compression ratio, evidence retention, state consistency, and cost, but they rarely define what a *correct* compression is.

This project implements a transparent proxy between the agent and the model API. V11 shifts the design goal from "make history shorter" to "continue working with verifiable state". Core mechanisms include reversible context folding (verbatim, addressable archive), commitment/identifier contracts, deterministic evidence probes, a per-session state table, per-model token calibration, a UCB compaction policy, cost attribution, and multi-tenant/multi-instance support. On the same 131-message, ∼9k-token coding session with the same summarizer, V11 retained 11/11 gold evidence spans and answered 4/4 reader questions at 0.86 compression, versus 0.64 evidence recall for both OpenClaw and Hermes. Its compaction cost was on par with OpenClaw (9.4k vs 8.9k tokens) and about 82% of Hermes, with roughly 38% of Hermes' latency.

The repository ships 88 automated tests (including real PostgreSQL integration, concurrent writes, and streaming pass-through), a CI matrix (Python 3.11–3.13 with a Postgres service), and reproducible evaluation scripts. We also state the limits: single-process caches, SQLite shared-volume constraints, in-process quota counters, and a small evaluation sample.

Contents

1 Introduction **3**
- 1.1 Problem . . . 3
- 1.2 Goals . . . 3
- 1.3 Contributions . . . 3

2 Survey: A Map of 109 Papers **3**
- 2.1 Method . . . 3
- 2.2 Findings . . . 4

3 Architecture **4**
- 3.1 Position in the Request Path . . . 4
- 3.2 Request Lifecycle . . . 4
- 3.3 Modules . . . 5

4 Mechanisms **5**
- 4.1 Reversible Folding . . . 5
- 4.2 Commitment Contracts and Evidence Probes . . . 6
- 4.3 State Table . . . 6

4.4 Policy and Calibration 6
4.5 Cost Attribution 7

5 Multi-Instance and Multi-Tenant 7
5.1 Tenant Isolation 7
5.2 Shared Storage 7
5.3 Quotas 7

6 Security and Operations 7

7 Evaluation 8
7.1 Three-Way Comparison 8
7.2 Paired Evaluation 8
7.3 Supplementary: LoCoMo Memory Benchmark (4 conversations) 8
7.4 Tests and Stress 8
7.5 Limitations 8

8 Engineering 9

9 Future Work 9

10 Conclusion 9

A Key Configuration 10

B Reproduction 10

C Selected References 10

1 Introduction

1.1 Problem

A coding or operations agent may run for dozens to hundreds of turns, and tool output—logs, files, web pages—dominates the history. The model context window (128k–1M tokens) caps what can be seen at once. Beyond it, requests fail, get truncated, or are summarized by the agent itself; summaries are unauditable, and constraints, file paths, and failing commands often disappear.

Three mechanisms dominate:

- **Agent-native compaction** (OpenClaw, Claude Code): tightly coupled to memory flushing, tool-pair integrity, and checkpoint replay; compaction happens inside the agent.
- **Runtime compressors** (Hermes): a standalone engine that protects head/tail and summarizes the middle with a cheaper model, triggered by API-reported token usage.
- **Engineered memory layers**: external stores plus retrieval, where a write policy decides what to remember.

Their common gap is that *correctness of compression is not verifiable.* Embedding similarity, summary length, and even downstream QA accuracy can hide a broken evidence chain [3, 4].

1.2 Goals

The proxy sits between the agent and the upstream API; the agent only points its `base_url` at a local port. V11 does not chase the highest compression ratio. It aims for:

1. **Reversibility**: folded history is always retrievable verbatim;
2. **Verifiability**: commitments and exact identifiers are checked after compaction and disclosed when missing;
3. **State consistency**: current file/decision state is distinguished from superseded state;
4. **Operability**: tenant isolation, quotas, encryption, retention, and cost attribution;
5. **Measurability**: paired continuation and evidence retention instead of embedding cosine.

1.3 Contributions

- A reversible folding protocol: typed `<folded_context>` placeholders, verbatim archive, pagination, encryption, and key rotation;
- Commitment contracts and evidence probes: deterministic scoring (commitments 45%, identifiers 25%, sections 15%, last ask 15%), with one MUST-PRESERVE retry below the floor;
- A session state table: file versions, stale-read marking, and decision supersession—directly targeting ghost memory;
- Multi-instance and multi-tenant support: tenant namespaces, a PostgreSQL session store, a Redis shared cache, and per-tenant quotas;
- Reproducible three-way and paired evaluations against OpenClaw and Hermes on the same model and transcript.

2 Survey: A Map of 109 Papers

2.1 Method

We queried the arXiv API with 12 topic searches (context compression, agent memory, KV cache, evaluation, RL policies, etc.) and 45 targeted title searches, sorted by submission date. After

deduplication, the candidate pool was 550 papers; a whitelist forced canonical works in and a keyword filter removed noise, yielding 109 papers (2023-10 to 2026-10). Data and scripts live under `research/`.

Table 1: Paper clusters (109 papers)

Cluster	Count	Representative work
Agent memory	25	MemGPT, Mem0, MemOS, A-MEM, MIRIX, Memory-R1
Context folding/compaction	19	ReFold, REMORY, MemTrace, Compact-Bench, KEEP-Bench
Prompt compression	18	LLMLingua/2, LongLLMLingua, RECOMP, ICAE, COMI
Evaluation	13	LongMemEval, LoCoMo, RULER, HELMET, NoLiMa
Retrieval	7	HippoRAG, Infinite Retrieval, Organize-then-Retrieve
Structured state	5	TEM, Narrative State Tracking
Long/streaming	5	Titans, MoBA, Quest
KV cache	4	SnapKV, MInference, Native Sparse Attention
RL policies	2	Memory-R2, Uni-Mem
Surveys	4	Context Engineering Survey, Agent Memory Survey
Other	6	—

2.2 Findings

1. **Summaries are being replaced by folding.** ReFold [1] shows training-free, reversible inter-turn folding beats predictive triggering; MemTrace [6] shows repository changes invalidate earlier evidence, so compaction must be state-consistent.
2. **Correctness equals commitment preservation.** "Compress the Context, Keep the Commitments" [2] defines context as a set of commitments; KEEP-Bench [3] shows QA metrics underestimate unfaithful compression.
3. **Evaluation must be paired.** Compact-Bench [4] compares full-history and summary-only continuations from the same repository state, avoiding false positives from lucky summaries.
4. **Flattening structure is a security hole.** Compaction Laundering [5] shows that collapsing typed conversations into plain text destroys provenance; folded placeholders must keep types and references.
5. **Evolving state is harder than fact recall.** [8, 7] name ghost memory: old, current, and transitional facts coexist and contaminate retrieval.
6. **With strong models, design less and retrieve more.** MemoryTax [9] and Skip the Compressor [10] show that when a model can search raw history itself, pre-built memory pipelines yield little; light folding plus searchable history often wins.

3 Architecture

3.1 Position in the Request Path

The proxy is transparent to the agent. It maintains side stores for archives, state, and sessions, and exposes health and metrics.

3.2 Request Lifecycle

1. **Tenant and identity**: middleware reads `X-Tenant-Id`; `extract_session_id` produces a tenant-scoped id (e.g., `acme:s1`) and echoes it in response headers.

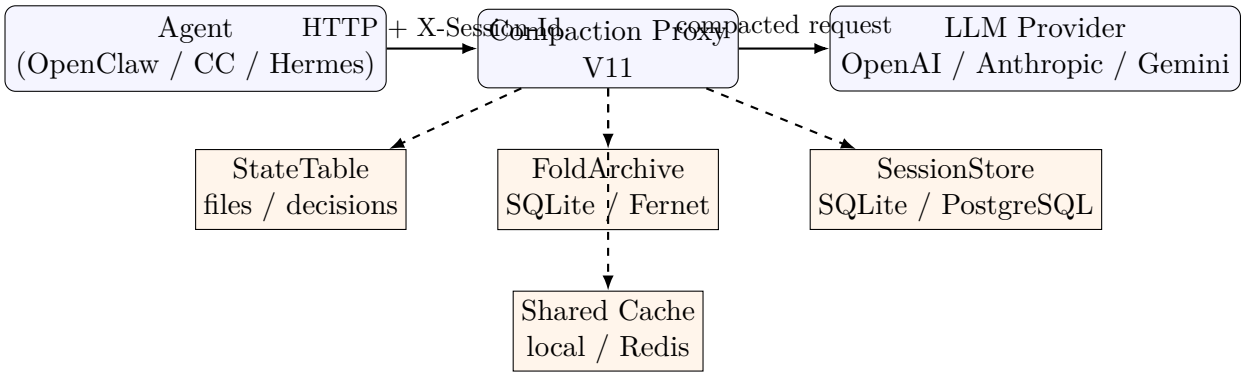


Figure 1: Proxy position and side stores

2. **Budget and trigger**: CJK-aware estimation (fast + accurate) corrected by a per-model factor; exceeding the effective threshold (default 80%, weak/strong/auto profiles) triggers proactive compaction; upstream overflow errors trigger recovery.
3. **Policy**: a UCB bandit keyed by task class and tenant chooses light/standard/deep, mapping to retained turns.
4. **Pipeline**: tool-output pruning → commitment/identifier extraction → submodular selection → parallel or single-block summary → fold archive → contract verification → evidence probes (retry once below 0.75).
5. **Assembly**: state snapshot + folded placeholder + sectioned summary + last six turns; tool-pair integrity is enforced before forwarding.
6. **Metering**: upstream `usage` is recorded for cost, savings, and latency; daily tenant budgets are updated.

3.3 Modules

Table 2: Core modules (tests and benchmark scripts excluded)

Module	LOC	Responsibility
`compaction-proxy.py`	8,273	HTTP service, pipeline, stores, routing
`folding.py`	517	archive, digest, placeholder, contract, encryption/rotation
`state_table.py`	304	file/decision state, stale marking, prune
`evidence.py`	111	four-dimensional evidence probes
`policy.py`	170	task classifier, UCB policy, capability profiles
`calibration.py`	68	per-model token calibration
`cost_tracker.py`	96	per-request ring buffer, daily quota
`tenant.py`	29	tenant context and namespacing
`shared_cache.py`	71	Redis/local compaction cache
`session_store_pg.py`	250	PostgreSQL session store

4 Mechanisms

4.1 Reversible Folding

Compaction no longer returns a free-text summary alone:

1. old messages are written *verbatim* to an archive (SQLite WAL), addressed as `fold:<hash>` where the hash includes tenant and content;
2. a deterministic digest is computed: message count, roles, tools, files, commands, errors, commitments;
3. a typed placeholder preserves provenance and a retrievable reference.

```
<folded_context ref="fold:6f775980e76a" messages="31" approx_tokens="8120">
[FOLD] 31 messages elided (user 14 / assistant 17)
[TOOLS] execx9, editx4, readx7
[FILES] /tmp/proj/src/core.py, /tmp/proj/config.yaml
[COMMANDS] pytest -q; npm test
[OBSERVED] 3 error/failure mentions
[COMMITMENTS] decision=2, constraint=1, task=1
[RETRIEVE] GET /fold/fold:6f775980e76a returns the original messages verbatim.
</folded_context>
```

The archive supports paginated retrieval (`offset/limit`, default 200, cap 500), optional Fernet encryption (key 0600), key rotation (`POST /maintenance/rotate-key`), and retention by tenant, age, and count.

4.2 Commitment Contracts and Evidence Probes

Before compaction, commitments (goals, constraints, decisions, errors, file operations, tasks) and exact identifiers (paths, function names, error fragments) are extracted. After compaction:

$$S = 0.45\, S_{\text{commit}} + 0.25\, S_{\text{id}} + 0.15\, S_{\text{sections}} + 0.15\, S_{\text{ask}}. \tag{1}$$

Below the floor $\tau = 0.75$, one retry is made with an explicit MUST-PRESERVE list; the better result is kept. Anything still missing is disclosed in `<unpreserved_commitments>` rather than silently lost.

4.3 State Table

Each session keeps a file table (path, last operation, timestamp, edit/read counts) and a decision table (goal/constraint/decision/task, active/superseded), rendered as:

```
<state_snapshot>
[FILES] /tmp/proj/config.yaml (edit 10:32; earlier reads stale)
[DECISIONS] active: (decision) use sqlite for cache ; superseded: 1
[NOTE] Treat earlier reads of edited files as stale; re-read before relying.
</state_snapshot>
```

This targets MemTrace-style consistency and A-TMA-style ghost memory.

4.4 Policy and Calibration

A UCB1 bandit selects an action per task class:

$$a_t = \arg\max_a \left(\bar{r}_a + \sqrt{\frac{2 \ln N}{n_a}} \right), \tag{2}$$

with a forced `light` action when the session's evidence EMA falls below the performance floor. Token calibration learns $f \leftarrow 0.8 f + 0.2$ (actual/estimated), clamped to $[0.5, 2.0]$, correcting CJK heuristics against the provider tokenizer.

4.5 Cost Attribution

Provider `usage` is preferred (OpenAI `prompt_tokens`, Anthropic `input_tokens`, Gemini `promptTokenCount`); the proxy records cost, savings, latency, cache hits, and tenant per compaction, falling back to estimation when usage is absent. `/health.cost_recent` aggregates by model and tenant with p50/p95 latency.

5 Multi-Instance and Multi-Tenant

5.1 Tenant Isolation

`X-Tenant-Id` is normalized into the request context; session ids become `tenant:session`, partitioning sessions, state, cost, and rate-limit buckets. Fold archives store `tenant_id`; cross-tenant retrieval returns 404; ref hashes include the tenant so identical content cannot overwrite another tenant's archive.

5.2 Shared Storage

- **PostgreSQL SessionStore**: sessions, prior summaries, transcripts, and ARC entries live in Postgres for cross-instance reads; search uses Postgres full-text with an ILIKE fallback; rows are trimmed per tenant.
- **Redis compaction cache**: with `COMPACTION_PROXY_REDIS_URL`, summaries use `SETEX` keyed by tenant and history hash; otherwise the process-local cache is used.
- **Instance identity**: `COMPACTION_PROXY_INSTANCE_ID` and `X-Instance-Id`; `MAINTENANCE_OWNER` restricts periodic cleanup to one instance.

5.3 Quotas

Per-tenant caps on archives, sessions, and daily tokens are enforced on write (oldest trimmed) and, with `ENFORCE_QUOTA=1`, by skipping compaction once the daily budget is exhausted—preventing one tenant from draining a shared upstream.

6 Security and Operations

Table 3: Security and operations

Capability	Implementation
Authentication	`REQUIRE_AUTH=1` refuses to start without an API secret (fail-closed); endpoints use `require_auth`
Rate limiting	Token bucket per `tenant:ip`; disabled by default, configurable per minute
Body size	Request cap (default 2 MB), 413 beyond
Encryption	Optional Fernet for folded payloads; key file 0600; rotation endpoint re-encrypts all rows
Retention	Age and row caps for folds/state/sessions; runs every 100 compactions; `/maintenance/prune` manual trigger
Observability	`/health` (storage, quotas, encryption, cost, p50/p95), `/metrics` (Prometheus)

7 Evaluation

7.1 Three-Way Comparison

On the same 131-message, ∼9,087-token coding session, with the same summarizer (local `tierflow_pro`) and the same four-question reader, we compared OpenClaw (prompt and cut policy reproduced from v2026.10.5), Hermes (the real `ContextCompressor`, aux LLM replaced by the same endpoint), and V11.

Table 4: Three-way compaction comparison (single run)

System	Compression	Evidence recall	State	QA	Cost (tokens)
OpenClaw	0.491	0.636	0.5	3/4	8,934
Hermes	0.471	0.636	1.0	3/4	11,410
NeverFull V11	**0.860**	**1.000**	**1.0**	**4/4**	9,369

Latency: OpenClaw 18.3 s, Hermes 67.7 s, V11 25.5 s. All three preserved decisions and paths, but only V11 preserved the failing-test command `pytest -q`—an evidence-chain difference, not merely an answer-accuracy one. OpenClaw did not mark the earlier config read as stale; Hermes was the most expensive and slowest.

7.2 Paired Evaluation

`benchmark/paired_eval.py` builds full, truncate, summary, and fold views of the same conversation and measures compression and evidence recall against identical probes. On synthetic data the fold view reaches 92.2% compression at 1.000 evidence recall; truncation reaches 65.0% at 1.000 (because answers sit at the tail), illustrating that the metric distinguishes *what survived*, not just length.

7.3 Supplementary: LoCoMo Memory Benchmark (4 conversations)

Beyond the coding session, we evaluated four long LoCoMo conversations (four category-stratified questions each, 16 total). With the same summarizer, a generic summary reaches 98.0% compression but retains only 0.199 evidence; V11 reaches 92.1% compression and retains 0.919 (on par with 65% truncation), with QA 0.167 (full-context ceiling 0.500; the local reader's temporal reasoning is the bottleneck). conv-41 summaries timed out twice and were skipped (resilience), so llm/v11 rows have views=3. Details: `research/EVAL-2026-10-26.md`.

Table 5: Supplementary LoCoMo evaluation (4 conversations x 4 questions)

Strategy	Compression	Evidence	QA	Summary time/conv
full	0.000	0.979	0.500	0 s
truncate	0.650	0.948	0.125	0 s
generic LLM summary	0.980	0.199	0.083	11.3 s
V11 fold	**0.921**	**0.919**	**0.167**	16.0 s

7.4 Tests and Stress

7.5 Limitations

- One synthetic conversation, four questions, one run; conclusions are directional and need LongMemEval/LoCoMo and multi-task paired sets.

Table 6: Verification results

Item	Result
Unit/integration tests	88 passing (6 real-PostgreSQL, concurrency, streaming pass-through, key rotation, quotas)
Concurrent writes	Before the fix: 78/320 and 116/320 errors under 8 threads; after locks and `busy_timeout`: 0
HTTP health	100 sequential p50 1 ms; 200 concurrent in 0.09 s, all successful
Error paths	Malformed JSON 400; 1.5 MB body 413; upstream down 502; process stays alive

- OpenClaw is a source-level reproduction, not a live Gateway session; its `keepRecentTokens` was scaled for the test.
- Cost uses measured `usage`; cache benefits and network overhead are not included.

8 Engineering

The V11 iteration produced eight main commits:

```
75893e9  P0+P1a: baseline, tests/CI, session-id, reversible folding
4ef1cb6  P1b-P5: state table, evidence, paired eval, policy, cost
b52d5b4  three-way benchmark (OpenClaw / Hermes / NeverFull)
9a53c9b  sqlite thread-safety fix + concurrency tests
8779806  ops pass: retention, auth/rate limits, encryption, calibration
d7a4574  cost tracker, key rotation, streaming test, docs
5f7cc44  tenant isolation, shared cache, instance identity
fba1a7a  PostgreSQL SessionStore, quotas, rotate-key endpoint
```

CI runs Python 3.11–3.13 with a Postgres service, byte-compiles entrypoints, runs 88 tests, and executes the paired evaluation. Dependencies are minimal: only `aiohttp` at runtime; Postgres, Redis, and semantic models are optional.

9 Future Work

1. **Scale evaluation**: add LongMemEval, LoCoMo, and Compact-Bench to CI with a real LLM reader and multi-task paired sets.
2. **Cache-aware compaction**: avoid rewriting the prefix every time and account for prompt-cache hits.
3. **Multi-instance consistency**: share policy/calibration in Redis and stress-test sticky routing and failover.
4. **Durable quotas**: persist daily budgets across restarts and export per-tenant billing.
5. **Learned policies**: replace the UCB bandit with a policy trained on task success and cost.

10 Conclusion

V11 moves context compression from "shorter summaries" to "verifiably continuing work": folding keeps the original reversible, contracts and probes verify commitments, the state table prevents stale evidence, and tenants/quotas make it operable. In a three-way test with the same model and comparable cost, it clearly led on evidence retention and task continuation. The project is still single-process-centric, but its mechanisms and evaluation harness are ready to scale.

A Key Configuration

Variable	Purpose
COMPACTION_PROXY_FOLDING	Enable reversible folding (default 0)
COMPACTION_PROXY_CONTRACT_RETRY	Retry on weak evidence (default 1)
COMPACTION_PROXY_EVIDENCE_FLOOR	Evidence floor (default 0.75)
COMPACTION_PROXY_POLICY	Enable UCB policy (default 0)
COMPACTION_PROXY_AGENT_PROFILE	weak/strong/auto threshold profile
COMPACTION_PROXY_STRICT_SESSION	Refuse content-fingerprint session ids
COMPACTION_PROXY_STORAGE	sqlite/postgres
COMPACTION_PROXY_PG_DSN	PostgreSQL DSN
COMPACTION_PROXY_REDIS_URL	Shared compaction cache
COMPACTION_PROXY_TENANT_MAX_FOLDS	Per-tenant archive cap
COMPACTION_PROXY_TENANT_MAX_SESSIONS	Per-tenant session cap
COMPACTION_PROXY_TENANT_DAILY_TOKENS	Per-tenant daily token budget
COMPACTION_PROXY_ENFORCE_QUOTA	Skip compaction when over budget
COMPACTION_PROXY_FOLD_ENCRYPT	Archive encryption
COMPACTION_PROXY_RETENTION_DAYS	Retention window
COMPACTION_PROXY_RATE_LIMIT	Requests per minute per IP
COMPACTION_PROXY_REQUIRE_AUTH	Fail closed without a secret

B Reproduction

```
# tests
python -m unittest discover -s tests -v

# paired evaluation
python benchmark/paired_eval.py --limit 3

# three-way comparison (requires a local OpenAI-compatible endpoint)
python benchmark/compare_systems.py --json research/data/compare.json

# run with folding + policy + encryption
COMPACTION_PROXY_FOLDING=1 COMPACTION_PROXY_FOLD_ENCRYPT=1 \
COMPACTION_PROXY_POLICY=1 COMPACTION_PROXY_AGENT_PROFILE=auto \
python compaction-proxy.py
```

C Selected References

References

[1] ReFold: Training-Free Reversible Inter-Turn Context Folding for Long-Horizon Agents. arXiv:2610.07863.

[2] Compress the Context, Keep the Commitments: A Formal Framework for Verifiable LLM Context Compression. arXiv:2605.17304.

[3] KEEP-Bench: Evaluating Evidence Preservation in Context Compression. arXiv:2610.26827.

[4] Compact-Bench: Measuring What Context Compaction Costs Long-Horizon Coding Agents. arXiv:2610.31710.

[5] Compaction Laundering: Provenance Destruction in Context-Compacting Coding Agents. arXiv:2610.29869.

[6] MemTrace: State-Consistent Memory for Long-Horizon Coding Agents. arXiv:2610.04838.

[7] A-TMA: Decoupling State-Aware Memory Failures in Long-Term Agent Memory. arXiv:2607.01935.

[8] Can Agent Memory Systems Track Evolving State? arXiv:2608.19652.

[9] MemoryTax: The Bitter Lesson of Agent Memory. arXiv:2610.29823.

[10] Skip the Compressor: Coding Agents Fix More Issues When They Pick Whole Files. arXiv:2610.21073.

[11] Prune the Summary, Not the History: Source-Judged Compaction for Long-Horizon Agents. arXiv:2610.20927.

[12] Maximising Transparency in Context Compaction Improves Retention of Facts and Commitments. arXiv:2610.20116.

[13] Persistent Context Graphs for Efficient Memory Compaction in LLM Agents. arXiv:2609.40118.

[14] Memory-R1: Enhancing Large Language Model Agents to Manage and Utilize Memories via Reinforcement Learning. arXiv:2508.19828.

[15] MemAgent: Reshaping Long-Context LLM with Multi-Conv RL-based Memory Agent. arXiv:2507.02259.

[16] Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory. arXiv:2504.19413.

[17] LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory. arXiv:2410.10813.

[18] Evaluating Very Long-Term Conversational Memory of LLM Agents. arXiv:2402.17753.

[19] RULER: What's the Real Context Size of Your Long-Context Language Models? arXiv:2404.06654.

[20] LLMLingua: Compressing Prompts for Accelerated Inference of Large Language Models. arXiv:2310.05736.